

Distributed Systems: From Theory to Practice

Impossibility Results Shape Design

The FLP impossibility result and CAP theorem from the distributed systems readings initially seemed abstract but became concrete through implementation. FLP proves that in asynchronous systems with potential failures, you cannot guarantee both safety and liveness in consensus. CAP states that during network partitions, you must choose between consistency and availability.

These constraints directly shaped assignments five and six. When deploying the Product API with multiple ECS instances behind an Application Load Balancer, I implemented a CP system: health checks remove unhealthy instances, sacrificing availability for consistency. An alternative AP design would serve stale cached data during partitions, maintaining availability at consistency's expense. Product descriptions tolerate eventual consistency; payment processing demands strong consistency.

Lamport's logical clocks paper revealed that distributed events have only partial ordering based on happened-before relationships. With multiple ECS instances, two simultaneous product updates arriving at different instances cannot be reliably ordered using local timestamps since clocks drift. Instance 1's clock running 100ms fast might timestamp a later request as earlier, causing incorrect last-write-wins conflict resolution.

Abstraction Layers and Coordination Costs

Assignment three's experiments quantified coordination costs across abstraction levels. Goroutine context switching takes 300 nanoseconds in user space. Process switching requires microseconds due to kernel involvement. Container-to-container communication involves millisecond network latency. Network partitions last hours. Each abstraction level trades convenience for performance.

The threading experiments showed specialized data structures outperform generic synchronization. When fifty goroutines wrote to a shared map, `sync.Map` completed in 5.7 milliseconds while mutex-protected maps required 11.5 milliseconds. Without synchronization, the program crashed. Buffered file I/O proved eleven times faster than unbuffered writes by batching system calls. These microbenchmarks revealed that coordination overhead exists at every level and optimization requires choosing appropriate mechanisms.

Information Hiding Enables Independent Evolution

Parnas's modularity paper demonstrated that modules should hide design decisions likely to change, not mirror processing steps. A naive KWIC design creates modules for input, circular shift, alphabetization, and output. Changing storage format requires modifying every module. The alternative encapsulates each decision: line storage hides format, circular shift hides whether shifts are precomputed or calculated on-demand. Implementation changes become localized.

This applied directly to the Product API. Storing products in sync.Map was one encapsulated decision. Checking exactly one hundred products per search was another. Properly hiding these decisions means switching from in-memory storage to PostgreSQL would require changing only the storage module. In microservices, this becomes critical—services must hide implementation behind stable APIs to enable independent evolution.

MapReduce: Parallel Computation Patterns

MapReduce excels at embarrassingly parallel problems where data partitions independently. Word counting processes each document independently until final aggregation. Assignment four's splitter-mapper-reducer pipeline demonstrated this across ECS tasks communicating through S3.

However, MapReduce cannot solve problems requiring strong consistency. Product search with consistent ranking or inventory updates need coordination. Partitioning products across nodes for parallel search requires either sharing global statistics or expensive global reranking. Assignment three's atomic counter experiments showed why: without coordination, concurrent updates create race conditions. Fifty goroutines incrementing without atomicity lost twelve to eighteen thousand updates. In distributed systems, this becomes the write-write conflict requiring locks or consensus.

Scaling Decisions Depend on Load Patterns

Assignment six revealed that scaling decisions require accurately understanding load patterns. The Product API handled three hundred requests per second on 0.25 vCPU with CPU below fifteen percent. Auto-scaling never triggered primarily because Locust's 0.1-0.5 second wait time limited effective request rate. Without wait time, even efficient code would face higher load and likely trigger scaling. Synthetic tests must model real user behavior including think time.

CloudWatch metrics guide optimization versus scaling decisions. Low CPU with plateauing throughput suggests bottlenecks elsewhere. High CPU indicates compute constraints. Horizontal scaling with ALB provides high availability since instance failures affect only fractional capacity, but introduces load balancing complexity and consistency challenges. Vertical scaling is simpler but creates single points of failure. For stateless services, horizontal scaling provides better availability and elastic cost efficiency.

Consistency Models and Coordination Costs

Strong consistency guarantees systems behave identically to non-replicated machines—all nodes see identical data simultaneously. This requires coordination: writes must be acknowledged by multiple nodes before committing, increasing latency. Assignment three's atomic operations demonstrated this tradeoff. Atomic increments guaranteed correct results; without atomicity, updates disappeared.

The Product API with multiple ECS instances provides eventual consistency—different instances might temporarily hold different data. This is acceptable for product descriptions but dangerous for inventory or payments. Linearizable consistency prevents double-booking but requires coordination through locks, Paxos, or single-leader architectures, all increasing latency. This tradeoff is fundamental: low latency with weak consistency, or strong consistency with higher latency.